

Навигация и картография робота по линиям

Восстановление 2D-траектории робота по видео и IMU

Перепечаев Виктор, Артур Адамов, Тимофей Вязовик
Ментор: Николай Кузнецов

Московский физико-технический институт

18 мая 2026 г.

Содержание

- 1 Команда
- 2 Постановка проблемы
- 3 Цель
- 4 Ресурсы и технологии
- 5 Календарное планирование
- 6 Реализация
- 7 Проблемы и их решения
- 8 Сравнительные результаты
- 9 Итоги
- 10 Дальнейшее развитие

- **Перепечаев Виктор**
- **Артур Адамов**
- **Тимофей Вязовик** — работает над статьёй (область работы полностью отдельная от нашей).
- **Ментор:** Николай Кузнецов

Распределение ролей — смешанное: оба участника (Виктор и Артур) были ответственны за все направления (CV, IMU, алгоритмы, инфраструктура), задачи перераспределялись по ситуации.

В самом начале семестра **один из участников команды отчислился**, что повлияло на распределение нагрузки и приоритизацию задач.

Постановка проблемы

Дано:

- видео с двух камер ZED — `Right_cam.mp4` и `Left_cam.mp4`,
- IMU-данные (акселерометр + гироскоп),
- timestamps кадров видео,
- Kalibr-калибровки IMU и camera-IMU.

Требуется: восстановить 2D-траекторию робота (x, y, yaw) во времени и визуализировать её.

- Робот движется по **синей ленте** на полу.
- Камера ZED содержит две монокулярные секции (LEFT и RIGHT): абсолютный масштаб напрямую из одной не восстанавливается, но два независимых наблюдения позволяют его уточнить.
- IMU — потребительский MEMS: gyro подходит для интегрирования yaw, accel для translation — нет ($\text{drift} \sim t^2$).
- Видео и IMU имеют независимые timestamps в наносекундах — нужна синхронизация по абсолютному времени.

Цель проекта

Цель

Создать воспроизводимый офлайн-пайплайн, который по записи видео и IMU строит 2D-траекторию робота.

Языки и инструменты:

- Python 3.10–3.14
- uv — менеджер зависимостей
- Beamer (x_eLaTeX) — презентация
- Git, GitHub Actions — CI

Computer Vision:

- OpenCV (HSV, distance transform, morphological operations)
- Plotly — интерактивные HTML-графики

Sensor fusion / Math:

- NumPy, SciPy
- IMU preintegration
- Kalibr (T_cam_imu, timeshift_cam_imu, AprilGrid)
- SVD, Procrustes, MAD-фильтр

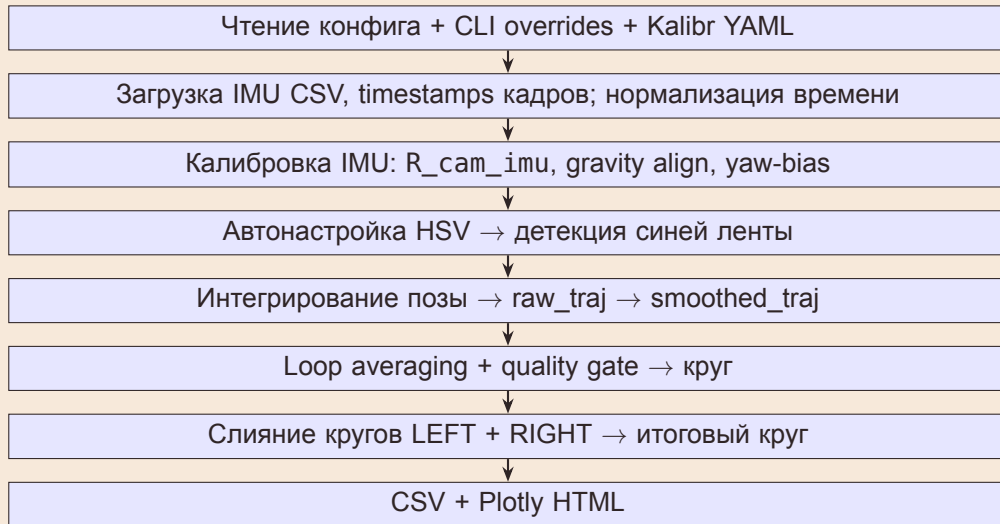
Качество:

- pytest, ruff, mypy
- Coverage badge в CI

Календарное планирование

Период	Активность
Февраль (1-я пол.)	Постановка задачи, инвентаризация данных, изучение Kalibr и формата SVO.
Февраль (2-я пол.)	Калибровка камеры (обход бага Kalibr с одной камерой), отладка ZED-stack.
Март (1-я пол.)	Калибровка IMU через AprilGrid, синхронизация времени видео и IMU.
Март (2-я пол.)	Базовое интегрирование yaw, gravity alignment, миграция инфраструктуры на uv.
Апрель (1-я пол.)	Реализация tape_line: HSV-маска, centerline, базовая интеграция позы.
Апрель (2-я пол.)	Офлайн-сглаживание, адаптивный confidence, variable speed correction.
Май (1-я пол.)	Loop averaging: нарезка по кругам, alignment, quality gate, ручные --lap--bounds.
Май (2-я пол.)	Использование второй камеры, fusion LEFT + RIGHT, тесты, CI, документация.

Общая схема пайплайна



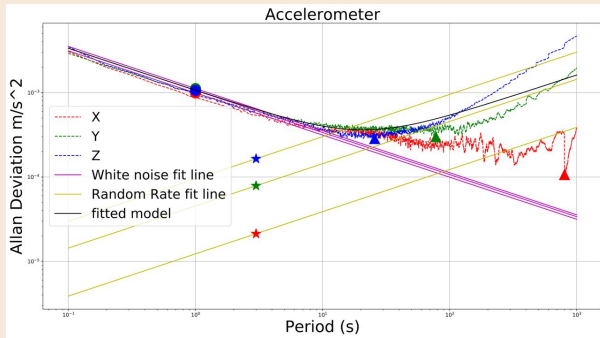
Измерение шумов IMU (Allan deviation)

Перед Kalibr-калибровкой IMU был измерен **Allan deviation** акселерометра по осям X, Y, Z.

По графику оцениваются:

- **White noise** (наклон $-1/2$) — σ_a плотность шума;
- **Random walk** (наклон $+1/2$) — $\sigma_{a,bias}$.

Эти параметры записываются в `imu-imu_calibration.yaml` и используются IMU preintegration.



Детекция ленты:

- Resize $\rightarrow$ HSV $\rightarrow$ inRange
- Largest connected component
- Median blur + morphological close
- Distance transform $\rightarrow$ centerline

Confidence по кадру:

- Число точек centerline
- Вертикальный span
- Площадь маски
- Валидность угла

Офлайн-сглаживание:

- Адаптивный порог confidence по распределению
- Centered weighted MA по angle_rad и bottom_x
- Окно задаётся в секундах, не кадрах

Интеграция позы:

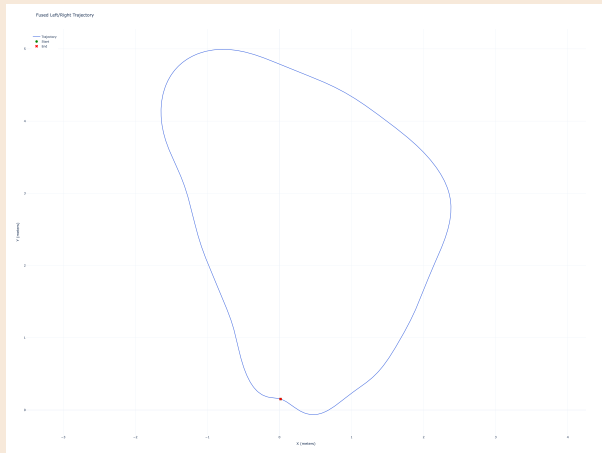
- $yaw \leftarrow yaw + \Delta yaw_{IMU} + correction$
- $\Delta x, \Delta y = v \cdot s_i \cdot dt \cdot (\cos, \sin)$
- lateral correction по bottom_x

Усреднение кругов (loop averaging)

- Оценка периода круга по визуальным дескрипторам кадров.
- Нарезка на круги: автоматически (anchor / по периоду) или вручную через `--lap-bounds`.
- Выравнивание кругов через rigid Procrustes (SVD без масштаба).
- Отброс плохих кругов: MAD-фильтр выбросов + quality gate по RMSE.
- Итоговый круг — лучший представитель со сглаживанием.

Слияние LEFT + RIGHT

- Каждая камера строит свой круг независимо.
- LEFT выравнивается к RIGHT (rigid Procrustes + phase search).
- Итог — точечное среднее двух кругов.
- Два независимых наблюдения снижают variance ошибки.



Итоговая траектория — усреднение LEFT и RIGHT.

Проблема 1: третий круг идёт против часовой стрелки

Симптом: 1-й и 2-й круги CW, 3-й — CCW; автодетекция направления обхода не различает.

Решение: CLI-флаг `--lap-bounds` для явного указания границ кругов.

Проблема 2: робот стоит первые 11 секунд

Симптом: стационарный старт сбивает anchor-search; найденные границы кругов сдвигаются относительно реальных.

Решение: ручные —lap—bounds с явным указанием начала первого круга в 11 с.

Проблема 3: IMU translation drift

Симптом: двойное интегрирование accel даёт ошибку $\sim t^2$ — через один круг сопоставимо с самим кругом.

Решение: yaw из IMU (один интеграл, drift медленный), translation — из априорного forward_speed_mps с variable-speed correction.

Проблема 4: Kalibr не работает с одной камерой

Симптом: intrinsic-калибровка падает с Python-исключением — библиотека рассчитана на multi-camera setup.

Источник: [ethz-asl/kalibr #364](#); проверка связности графа в `MulticamGraph.py`.

Решение (из community):

```
sed -i 's/return self.G.adhesion()/return True/g' \
    .../kalibr_camera_calibration/MulticamGraph.py
```

Проблема 5: IMU-калибровка и инфраструктура

- Kalibr + AprilGrid: требовалось физически вращать робота по всем осям, чтобы оценить параметры IMU.
- Ни у кого в команде не было ноутбука с NVIDIA GPU — часть инструментов (SVO2 от ZED) требует CUDA.
- Привлекали сторонних людей: Роберт Вахненко предоставлял свой ноутбук для конвертации SVO2.

Сравнение вариантов прогона (одна камера vs две)

Вариант	Длина (м)	Bbox (м)	Aspect
RIGHT auto (3 крыга, full)	11,32	3,42 × 3,80	1,11
RIGHT --lap-bounds	12,70	3,04 × 4,57	1,51
LEFT --lap-bounds	10,89	3,00 × 4,10	1,37
FUSED LEFT + RIGHT	11,22	2,93 × 4,27	1,46

Качество выравнивания LEFT → RIGHT:

- Mean residual: **0,34 м** (~8% от bbox)
- Median: 0,28 м, p95: 0,78 м
- Reflection не нужна — обе камеры дают траекторию в одной ориентации

Что получено

- Воспроизводимый офлайн-пайплайн Навигация и картография робота по линиям с использованием двух камер ZED.
- Замкнутый круг по записи с несколькими повторениями, усреднённый по LEFT + RIGHT.
- CI с тестами, типизацией и линтером; coverage badge.

Чему научились

- Sensor fusion в условиях ограниченного бюджета (monocular + MEMS IMU).
- Робастные методы (MAD, quality gates, адаптивные пороги).
- Инженерные практики: uv, CI, отладка библиотек уровня Kalibr.

Алгоритм:

- Pose graph optimization для коррекции дрейфа.
- Bundle adjustment с camera intrinsics.

Сенсоры:

- Stereo / depth → метрический масштаб без `forward_speed`.
- Wheel odometry как дополнительный prior.

Функционал:

- Чтение стрелочных символов на поле.
- Сценарии «следуй за объектом X».
- Тестирование в разных условиях освещения.
- Интеграция с прошлогодним ROS-стеком.

Спасибо за внимание!